3841. 查询树上回文路径

难度: **Hard**

给你一棵包含 `n` 个节点的无向树，节点编号从 0 到 `n - 1`。树由一个长度为 `n - 1` 的二维数组 `edges` 表示，其中 `edges[i] = [u_i, v_i]` 表示节点 `u_i` 和 `v_i` 之间存在一条无向边。

另给你一个长度为 `n` 且由小写英文字母组成的字符串 `s`，其中 `s[i]` 表示分配给节点 `i` 的字符。

还给你一个字符串数组 `queries`，其中每个 `queries[i]` 为以下形式之一：

- `"update u_i c"`：将节点 `u_i` 处的字符更改为 `c`。正式地，更新 `s[u_i] = c`。
- `"query u_i v_i"`：判断从 `u_i` 到 `v_i` 的 **唯一** 路径（含两端点）上的字符所组成的字符串，是否可以 **重新排列** 成一个 **回文串**。

返回一个布尔数组 `answer`，如果第 `i` 个类型为 `"query u_i v_i"` 的查询可以重新排列成 **回文串**，则 `answer[i]` 为 `true`，否则为 `false`。

回文串 是指正读和反读都相同的字符串。

示例 1：

输入： n = 3, edges = [[0,1],[1,2]], s = "aac", queries = ["query 0 2","update 1 b","query 0 2"]

输出： [true,false]

解释：

- `"query 0 2"`：路径 `0 → 1 → 2` 得到的字符串是 `"aac"`，可以重新排列形成 `"aca"`，这是一个回文串。因此，`answer[0] = true`。
- `"update 1 b"`：将节点 1 更新为 `'b'`，现在 `s = "abc"`。
- `"query 0 2"`：路径上的字符为 `"abc"`，无法重新排列形成回文串。因此，`answer[1] = false`。

因此，`answer = [true, false]`。

示例 2：

输入： n = 4, edges = [[0,1],[0,2],[0,3]], s = "abca", queries = ["query 1 2","update 0 b","query 2 3","update 3 a","query 1 3"]

输出： [false,false,true]

解释：

- `"query 1 2"`：路径 `1 → 0 → 2` 得到的字符串是 `"bac"`，无法重新排列形成回文串。因此，`answer[0] = false`。
- `"update 0 b"`：将节点 0 更新为 `'b'`，现在 `s = "bbca"`。
- `"query 2 3"`：路径 `2 → 0 → 3` 得到的字符串是 `"cba"`，无法重新排列形成回文串。因此，`answer[1] = false`。
- `"update 3 a"`：将节点 3 更新为 `'a'`，`s = "bbca"`。
- `"query 1 3"`：路径 `1 → 0 → 3` 得到的字符串是 `"bba"`，可以重新排列形成 `"bab"`，这是一个回文串。因此，`answer[2] = true`。

因此，`answer = [false, false, true]`。

提示：

- `1 <= n == s.length <= 5 * 10^4`
- `edges.length == n - 1`
- `edges[i] = [u_i, v_i]`
- `0 <= u_i, v_i <= n - 1`
- `s` 由小写英文字母组成。
- 输入生成的 `edges` 表示一棵有效的树。
- `1 <= queries.length <= 5 * 10^4`
  - `queries[i] = "update u_i c"` 或
  - `queries[i] = "query u_i v_i"`
  - `0 <= u_i, v_i <= n - 1`
  - `c` 是一个小写英文字母。